

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10

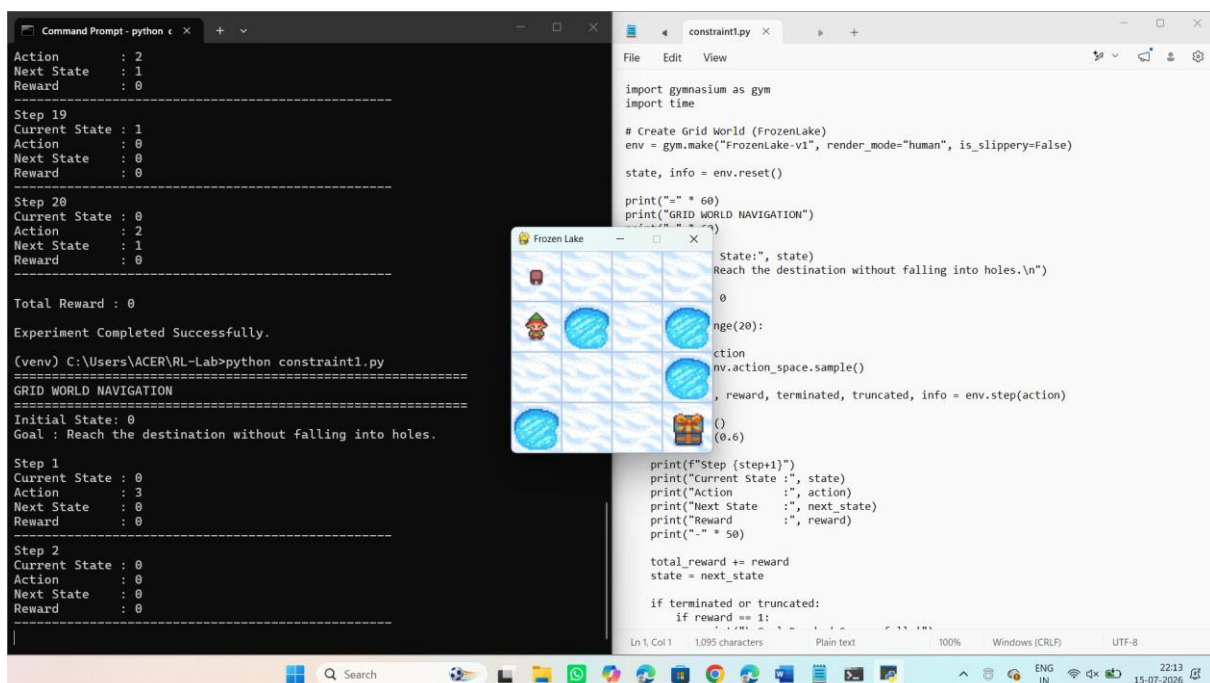
Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.



```
Command Prompt - python < + v
Action : 2
Next State : 1
Reward : 0
-----
Step 19
Current State : 1
Action : 0
Next State : 0
Reward : 0
-----
Step 20
Current State : 0
Action : 2
Next State : 1
Reward : 0
-----
Total Reward : 0
Experiment Completed Successfully.
(venv) C:\Users\ACER\RL-Lab>python constraint1.py
=====
GRID WORLD NAVIGATION
=====
Initial State: 0
Goal : Reach the destination without falling into holes.

Step 1
Current State : 0
Action : 3
Next State : 0
Reward : 0
-----
Step 2
Current State : 0
Action : 0
Next State : 0
Reward : 0
-----

constraint1.py
File Edit View
import gymnasium as gym
import time

# Create Grid World (FrozenLake)
env = gym.make("FrozenLake-v1", render_mode="human", is_slippery=False)

state, info = env.reset()

print("-" * 60)
print("GRID WORLD NAVIGATION")
print("-" * 60)

State:", state)
Reach the destination without falling into holes.\n")
0
nge(20):
action
nv.action_space.sample()
, reward, terminated, truncated, info = env.step(action)
()
(0.6)

print(f"Step {step+1}")
print("Current State :", state)
print("Action : ", action)
print("Next State : ", next_state)
print("Reward : ", reward)
print("-" * 50)

total_reward += reward
state = next_state

if terminated or truncated:
    if reward == 1:
        print("Goal Reached! Total Reward: ", total_reward)
    else:
        print("Game Over! Total Reward: ", total_reward)
    break

Ln 1, Col 1 1,095 characters Plain text 100% Windows (CRLF) UTF-8
22:13 15-07-2026
```

```

Agent Failed (Hit Hole or Episode Ended).
Total Reward : 0
Experiment Completed Successfully.

(venv) C:\Users\ACER\RL-Lab>python constraint1.py

=====
GRID WORLD NAVIGATION
=====
Initial State: 0
Goal : Reach the destination without falling into holes.

Step 1
Current State : 0
Action       : 3
Next State   : 0
Reward       : 0

Step 2
Current State : 0
Action       : 2
Next State   : 1
Reward       : 0

Step 3
Current State : 1
Action       : 5
Next State   : 5
Reward       : 0

Agent Failed (Hit Hole or Episode Ended).
Total Reward : 0
Experiment Completed Successfully.

(venv) C:\Users\ACER\RL-Lab>

```

```

constraint1.py
File Edit View

import gymnasium as gym
import time

# Create Grid World (FrozenLake)
env = gym.make("FrozenLake-v1", render_mode="human", is_slippery=False)

state, info = env.reset()

print("=" * 60)
print("GRID WORLD NAVIGATION")
print("=" * 60)

print("Initial State:", state)
print("Goal : Reach the destination without falling into holes.\n")

total_reward = 0

for step in range(20):

    # Random action
    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    env.render()
    time.sleep(0.6)

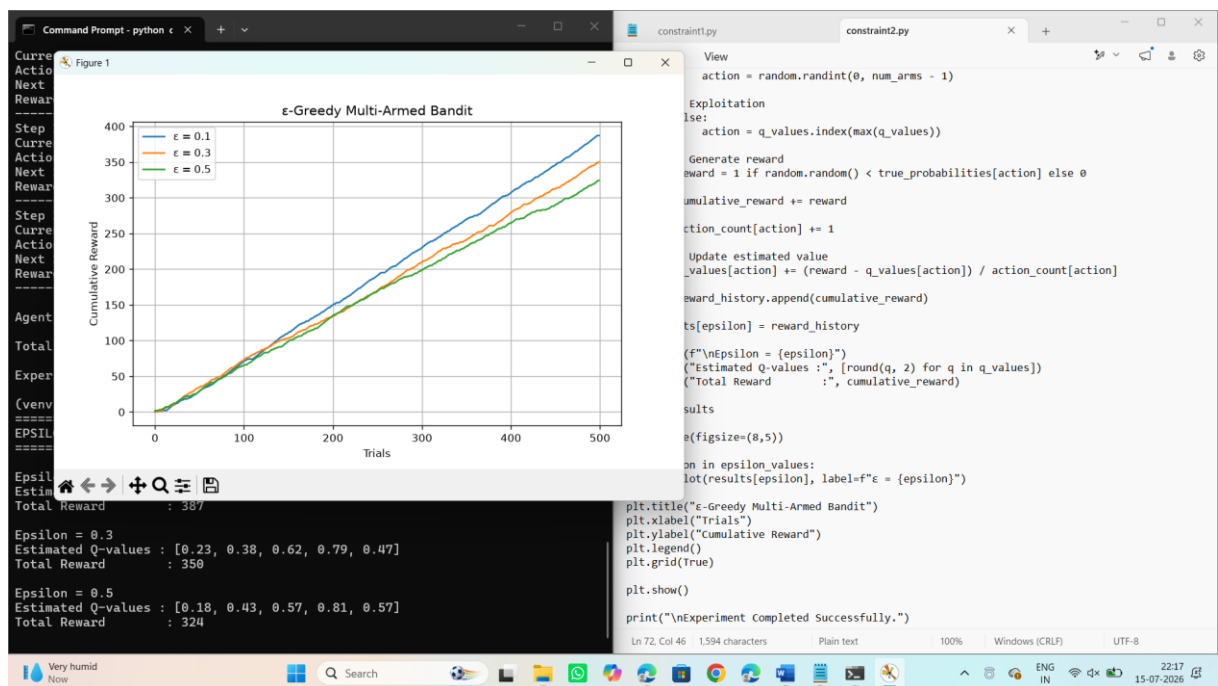
    print(f"Step {step+1}")
    print("Current State :", state)
    print("Action       :", action)
    print("Next State   :", next_state)
    print("Reward       :", reward)
    print("-" * 50)

    total_reward += reward
    state = next_state

    if terminated or truncated:
        if reward == 1:
            print("Goal Reached!")
        else:
            print("Agent Failed!")
        break

```

- Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.



```

Command Prompt
Reward : 0
-----
Step 2
Current State : 0
Action : 2
Next State : 1
Reward : 0
-----
Step 3
Current State : 1
Action : 1
Next State : 5
Reward : 0
-----
Agent Failed (Hit Hole or Episode Ended).
Total Reward : 0
Experiment Completed Successfully.

(venv) C:\Users\ACER\RL-Lab>python constraint2.py
=====
EPSILON-GREEDY MULTI-ARMED BANDIT
=====
Epsilon = 0.1
Estimated Q-values : [0.23, 0.47, 0.5, 0.83, 0.25]
Total Reward : 387

Epsilon = 0.3
Estimated Q-values : [0.23, 0.38, 0.62, 0.79, 0.47]
Total Reward : 350

Epsilon = 0.5
Estimated Q-values : [0.18, 0.43, 0.57, 0.81, 0.57]
Total Reward : 324
Experiment Completed Successfully.

(venv) C:\Users\ACER\RL-Lab>

constraint2.py
# Constraint-Based Problem 2
# Epsilon-Greedy Multi-Armed Bandit

import random
import matplotlib.pyplot as plt

# Probability of reward for each arm
true_probabilities = [0.2, 0.4, 0.6, 0.8, 0.5]

num_arms = len(true_probabilities)
trials = 500

epsilon_values = [0.1, 0.3, 0.5]

results = {}

print("=" * 65)
print("EPSILON-GREEDY MULTI-ARMED BANDIT")
print("=" * 65)

for epsilon in epsilon_values:
    q_values = [0] * num_arms
    action_count = [0] * num_arms

    cumulative_reward = 0
    reward_history = []

    for _ in range(trials):
        # Exploration
        if random.random() < epsilon:
            action = random.randint(0, num_arms - 1)
        # Exploitation
        else:
            action = q_values.index(max(q_values))

        # Generate reward
    
```

- Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

```

Command Prompt
Experiment Completed Successfully.

(venv) C:\Users\ACER\RL-Lab>python constraint3.py
=====
MARKOV DECISION PROCESS (MDP)
=====
States : ['Start', 'Path A', 'Path B', 'Goal']
Energy Available : 15

Robot Starting...

Step 1
Current State : Start
Action : Right
Next State : Path A
Reward : -2
Remaining Energy : 13
-----
Step 2
Current State : Path A
Action : Right
Next State : Path B
Reward : -2
Remaining Energy : 11
-----
Step 3
Current State : Path B
Action : Right
Reached Goal!
Next State : Goal
Reward : 100
Remaining Energy : 9
-----
Final State : Goal
Total Reward : 96
Energy Left : 9
Experiment Completed Successfully.

(venv) C:\Users\ACER\RL-Lab>

constraint3.py
# Constraint-Based Problem 3
# Markov Decision Process (MDP) for Autonomous Robot

# States
states = ["Start", "Path A", "Path B", "Goal"]

# Actions
actions = ["Right", "Right", "Right"]

# Initial state
current_state = 0

# Energy Constraint
energy = 15

# Reward
total_reward = 0

print("=" * 65)
print("MARKOV DECISION PROCESS (MDP)")
print("=" * 65)

print("States :", states)
print("Energy Available :", energy)

print("\nRobot Starting...\n")

for step, action in enumerate(actions, start=1):
    if energy <= 0:
        print("Battery Exhausted!")
        break

    print(f"Step {step}")
    print("Current State :", states[current_state])
    print("Action :", action)

    # Move to next state
    current_state += 1
    
```

- Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

```
Command Prompt
Next State : Path B
Reward : -2
Remaining Energy : 11
-----
Step 3
Current State : Path B
Action : Right
Reached Goal!
Next State : Goal
Reward : 100
Remaining Energy : 9
-----
Final State : Goal
Total Reward : 96
Energy Left : 9

Experiment Completed Successfully.

(venv) C:\Users\ACER\RL-Lab>python constraint4.py
=====
VALUE ITERATION USING BELLMAN EQUATION
=====

Optimal State Values

[[ 72.9  81.  90. ]
 [ 81.   0. 100. ]
 [ 90. 100.   0. ]]

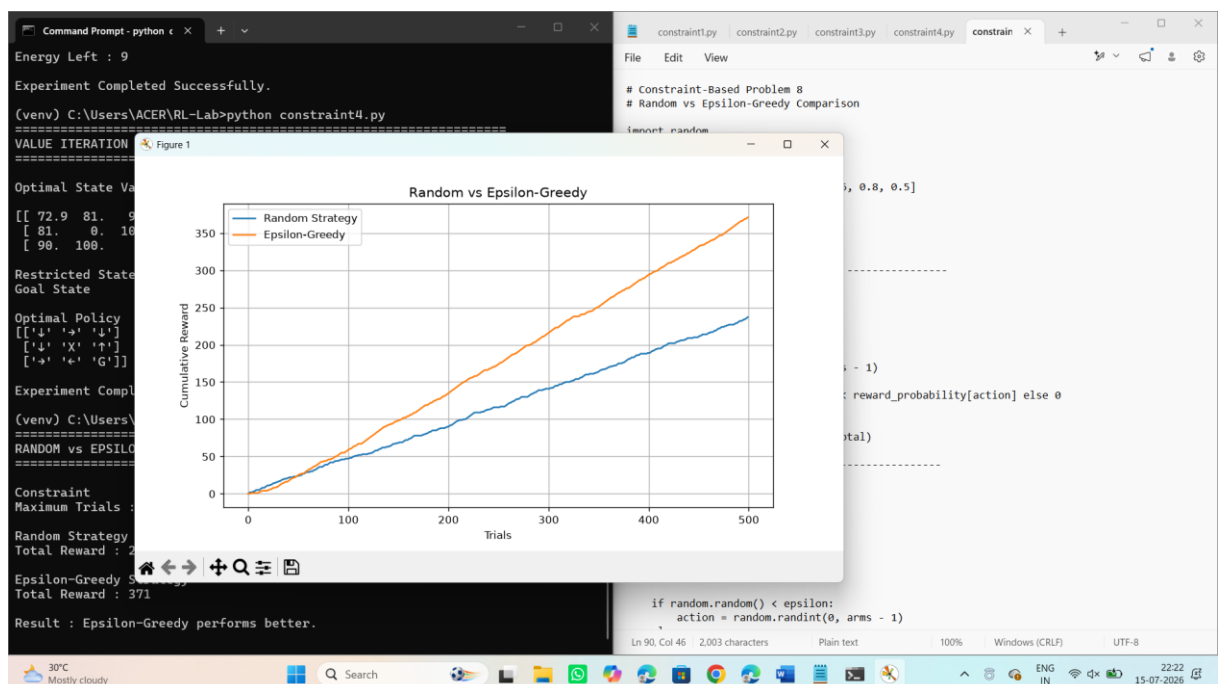
Restricted State : (1,1)
Goal State : (2,2)

Optimal Policy
[['↓' '→' '↓']
 ['↓' 'X' '↑']
 ['→' '←' 'G']]

Experiment Completed Successfully.

(venv) C:\Users\ACER\RL-Lab>
```

8. Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.



```
Command Prompt
(venv) C:\Users\ACER\RL-Lab>python constraint4.py
=====
VALUE ITERATION USING BELLMAN EQUATION
=====
Optimal State Values
[[ 72.9  81.  90. ]
 [ 81.   0. 100. ]
 [ 90. 100.   0. ]]
Restricted State : (1,1)
Goal State      : (2,2)
Optimal Policy
[['↓' '→' '↓']
 ['↓' 'X' '↓']
 ['→' '←' 'G']]
Experiment Completed Successfully.
(venv) C:\Users\ACER\RL-Lab>python constraint8.py
=====
RANDOM vs EPSILON-GREEDY COMPARISON
=====
Constraint
Maximum Trials : 500
Random Strategy
Total Reward : 237
Epsilon-Greedy Strategy
Total Reward : 371
Result : Epsilon-Greedy performs better.
Experiment Completed Successfully.
(venv) C:\Users\ACER\RL-Lab>
```

```
constraint1.py constraint2.py constraint3.py constraint4.py constrain
File Edit View
# Constraint-Based Problem 8
# Random vs Epsilon-Greedy comparison
import random
import matplotlib.pyplot as plt
# Reward probability of each arm
reward_probability = [0.2, 0.4, 0.6, 0.8, 0.5]
arms = len(reward_probability)
trials = 500
epsilon = 0.1
# ----- Random Strategy -----
random_rewards = []
random_total = 0
for _ in range(trials):
    action = random.randint(0, arms - 1)
    reward = 1 if random.random() < reward_probability[action] else 0
    random_total += reward
    random_rewards.append(random_total)
# ----- Epsilon-Greedy -----
q_values = [0] * arms
counts = [0] * arms
epsilon_rewards = []
epsilon_total = 0
for _ in range(trials):
    if random.random() < epsilon:
        action = random.randint(0, arms - 1)
    else:
        action = np.argmax(q_values)
```

Ln 90, Col 46 2,003 characters Plain text 100% Windows (CRLF) UTF-8 22:22 15-07-2028

Code of Conduct:

I _S.Sirivally / 192472371_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided